WEEK-2:

TASK-1:-

```
 GNU nano 8.7.1                                  task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <string.h>

int main()
{
    char command[100];
    pid_t pid;

    printf("Enter a Linux command: ");
    scanf("%s", command);

    pid = fork();

    if(pid < 0)
    {
        printf("Fork Failed!\n");
        return 1;
    }

    else if(pid == 0)
    {
        printf("\nChild Process\n");
        printf("Child PID : %d\n", getpid());
        printf("Parent PID: %d\n", getppid());

        printf("\nExecuting command: %s\n\n", command);

        execlp(command, command, NULL);

        printf("Command execution failed.\n");
    }

    else
    {
        printf("\nParent Process\n");
        printf("Parent PID: %d\n", getpid());
        printf("Child PID : %d\n", pid);

        wait(NULL);

        printf("\nChild process completed.\n");
    }

    return 0;
}

^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo      M-A Set Mark   M-] To Bracket  M-P Previous  ^B Back       ^Left Prev Word  ^A Home
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^/ Go To Line M-E Redo      M-6 Copy       ^Q Where Was    M-N Next      ^F Forward    ^Right Next Word ^E End
```

```
surajp@LAPTOP-6E22AR6C:~$ nano task1.c
surajp@LAPTOP-6E22AR6C:~$ gcc task1.c -o task1
surajp@LAPTOP-6E22AR6C:~$ ./task1
Enter a Linux command: ls

Parent Process
Parent PID: 2306
Child PID : 2307

Child process
Child PID : 2307
Parent PID: 2306

Executing command: ls
OSSP            file1.text    fork_example.c   task1     task5
backup.txt      file1.txt     hello.c          task1.c   task5.c
exec_example    file2.txt     hello.c          task1
exec_example.c  fork_example  process.c        task1.c

Child process completed.
surajp@LAPTOP-6E22AR6C:~$ pwd
/home/surajp
surajp@LAPTOP-6E22AR6C:~$ ./task1
Enter a Linux command: pwd

Parent Process
Parent PID: 2329
Child PID : 2330

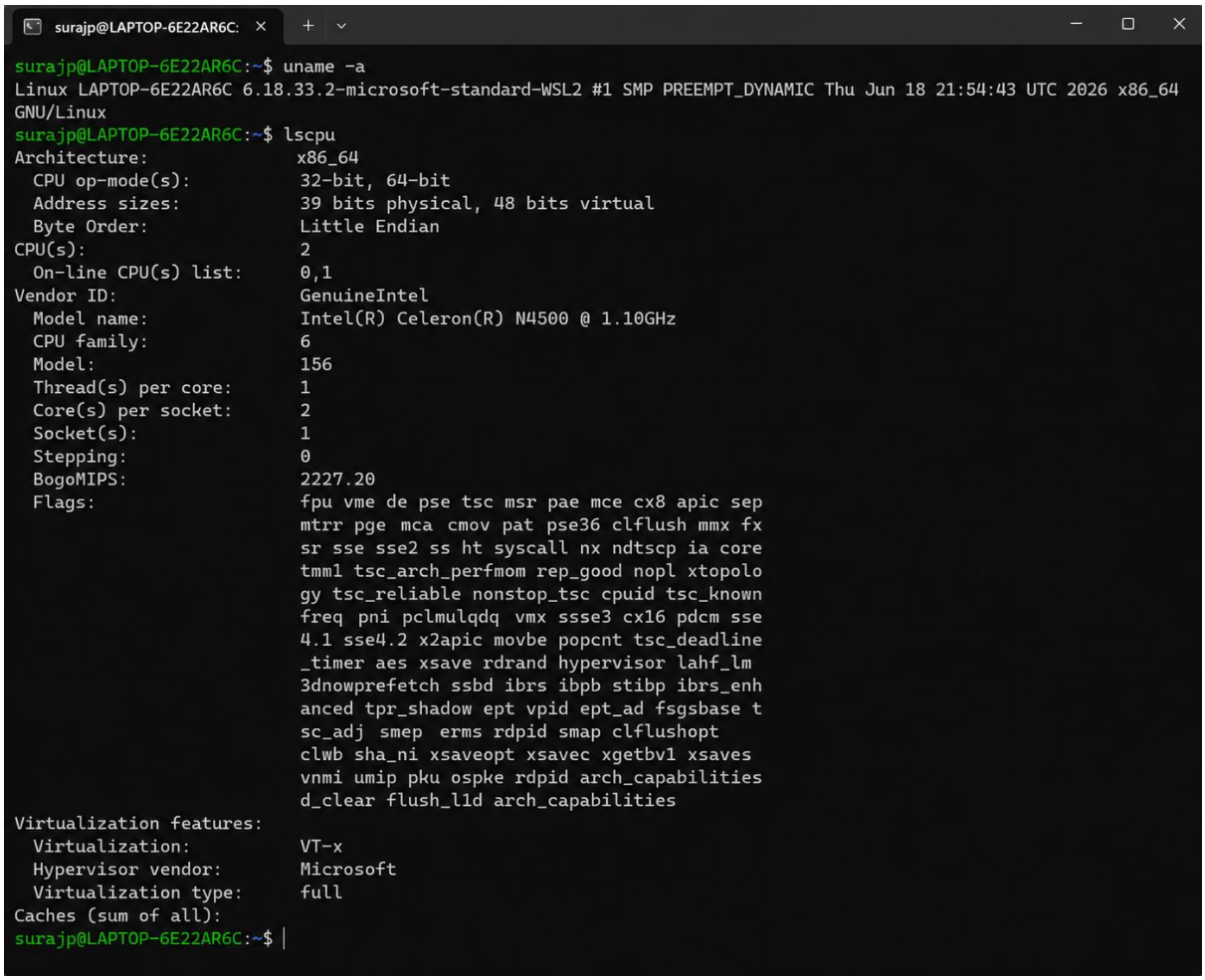
Child process
Child PID : 2330
Parent PID: 2329

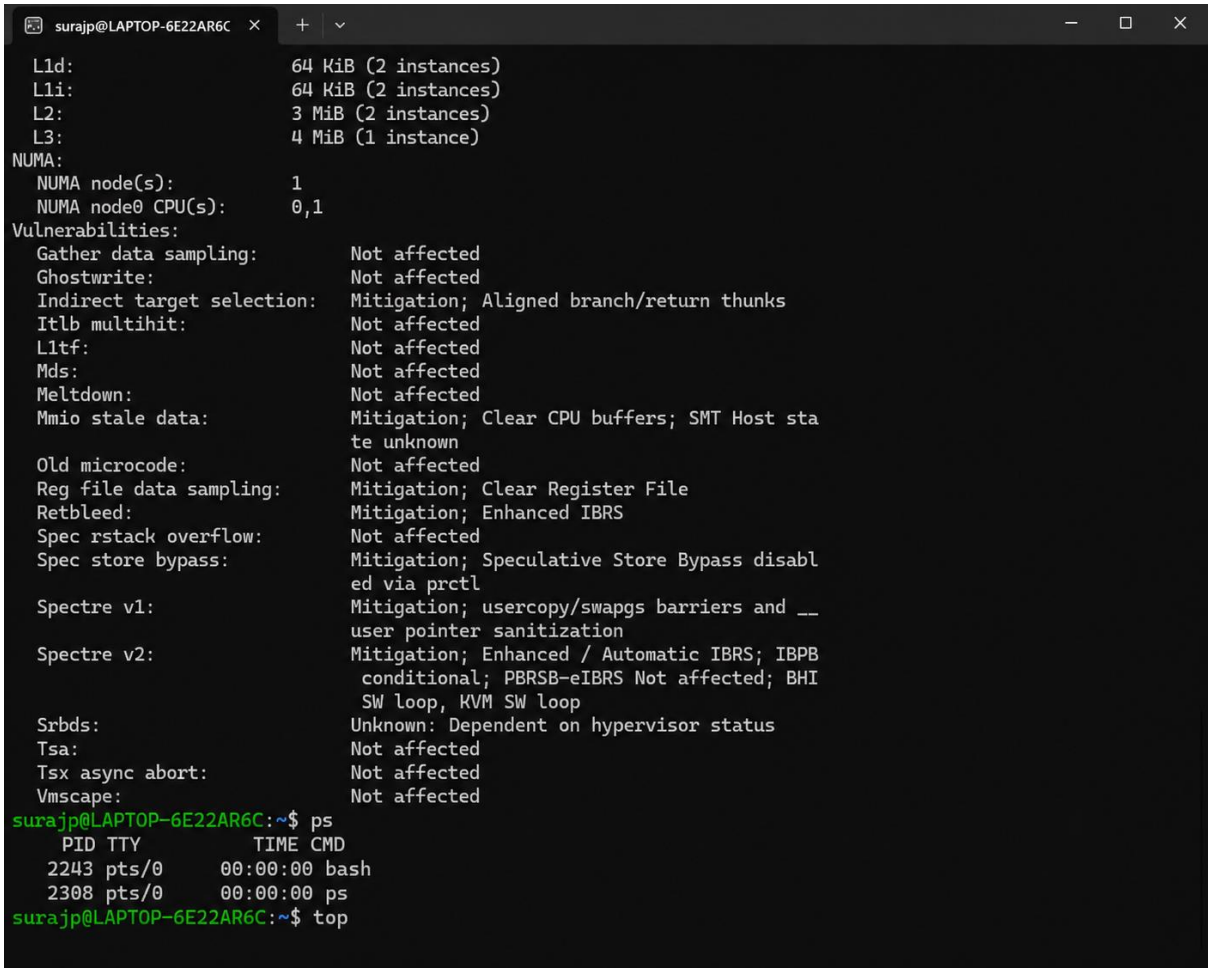
Executing command: pwd
/home/surajp

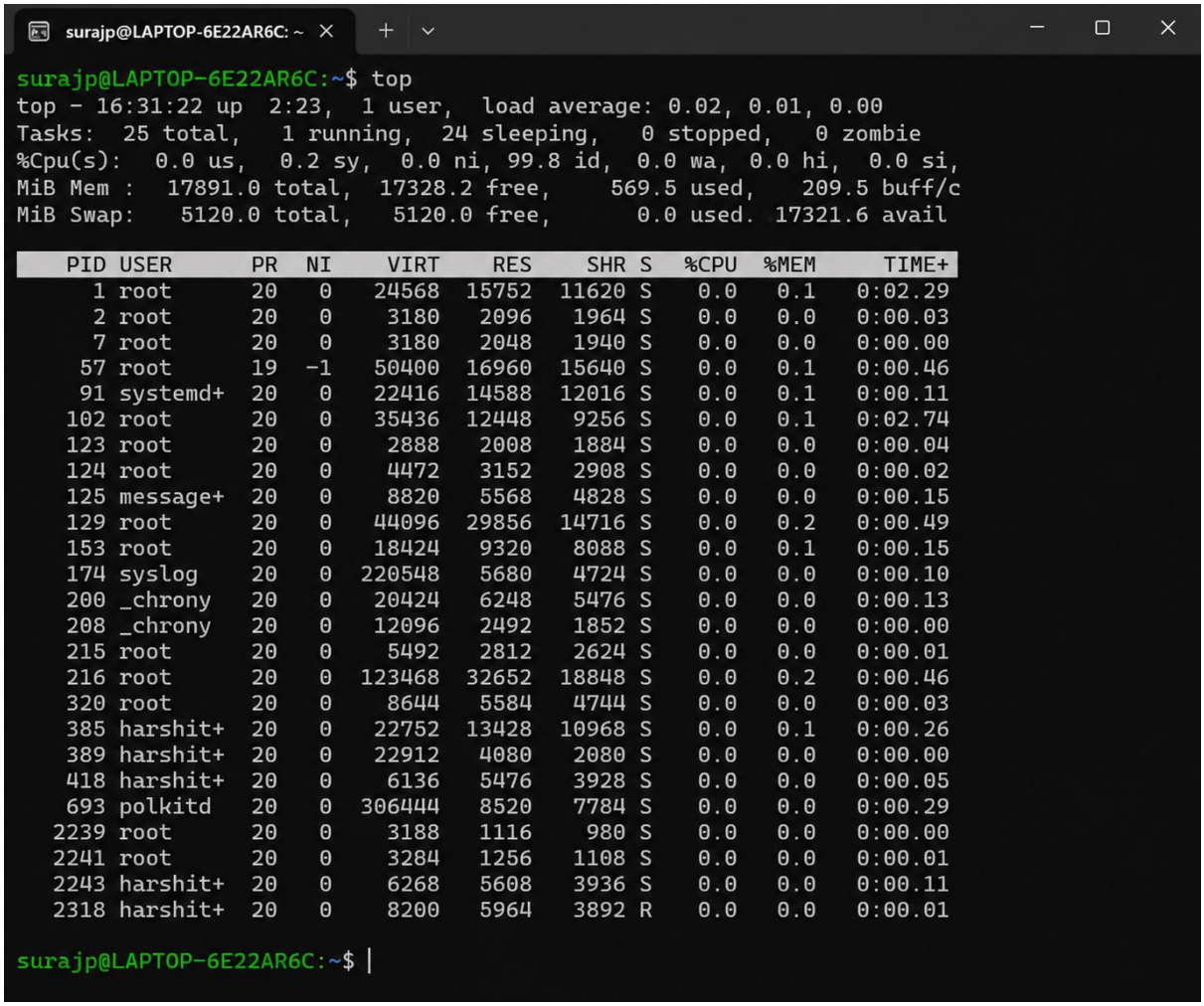
Child process completed.
surajp@LAPTOP-6E22AR6C:~$
```

This program demonstrates process creation using fork(), command execution using execlp(), and process synchronization using wait(). It also shows how parent and child processes execute independently while sharing a parent-child relationship.

Task-2:-




surajp@LAPTOP-6E22AR6C
L1d: 64 KiB (2 instances)
L1i: 64 KiB (2 instances)
L2: 3 MiB (2 instances)
L3: 4 MiB (1 instance)
NUMA:
NUMA node(s): 1
NUMA node0 CPU(s): 0,1
Vulnerabilities:
Gather data sampling: Not affected
Ghostwrite: Not affected
Indirect target selection: Mitigation; Aligned branch/return thunks
Itlb multihit: Not affected
L1tf: Not affected
Mds: Not affected
Meltdown: Not affected
Mmio stale data: Mitigation; Clear CPU buffers; SMT Host state unknown
Old microcode: Not affected
Reg file data sampling: Mitigation; Clear Register File
Retbleed: Mitigation; Enhanced IBRS
Spec rstack overflow: Not affected
Spec store bypass: Mitigation; Speculative Store Bypass disabled via prctl
Spectre v1: Mitigation; usercopy/swapgs barriers and __user pointer sanitization
Spectre v2: Mitigation; Enhanced / Automatic IBRS; IBPB conditional; PBRSB-eIBRS Not affected; BHI SW loop, KVM SW loop
Srbds: Unknown: Dependent on hypervisor status
Tsa: Not affected
Tsx async abort: Not affected
Vmscape: Not affected
surajp@LAPTOP-6E22AR6C:~$ ps
PID TTY TIME CMD
2243 pts/0 00:00:00 bash
2308 pts/0 00:00:00 ps
surajp@LAPTOP-6E22AR6C:~$ top



The experiment demonstrated how Linux abstracts hardware resources by providing system services that efficiently manage CPU, memory, storage, and I/O devices. The terminal commands helped analyze system information, processor details, and running processes in real time.